```
"""
Telegram Marketplace Bot — Phase 1 (Foundation) + Phase 4 (Orders)
Single-file version: everything (config, database, keyboards, handlers) lives here.

Setup:
    pip install -r requirements.txt
    export BOT_TOKEN=your_token_here  (or put it in a .env file)
    export ADMIN_IDS=123456789  (your numeric Telegram user ID)
    python bot.py
"""

import os
import asyncio
import logging
```

```
import aiosqlite
from dotenv import load_dotenv

from aiogram import Bot,
Dispatcher, Router, F
from aiogram.client.default
import DefaultBotProperties
from aiogram.filters import
CommandStart
from aiogram.fsm.context import
FSMContext
from aiogram.fsm.state import
State, StatesGroup
from aiogram.fsm.storage.memory
import MemoryStorage
from aiogram.types import (
    Message, CallbackQuery,
    ReplyKeyboardMarkup,
KeyboardButton,
    InlineKeyboardMarkup,
InlineKeyboardButton,
)

logging.basicConfig(level=loggin
```

```
g.INFO)
load_dotenv()

#
==============================
========================
# CONFIG
#
==============================
========================

BOT_TOKEN =
os.getenv("BOT_TOKEN", "")
DB_PATH = os.getenv("DB_PATH",
"marketplace.db")

_raw_admins =
os.getenv("ADMIN_IDS", "")
ADMIN_IDS = {int(x.strip()) for
x in _raw_admins.split(",") if
x.strip().isdigit()}

if not BOT_TOKEN:
    raise
RuntimeError("BOT_TOKEN is not
```

```
set. Set it as an env var or in
a .env file.")

# Shown to buyers when they
check out. Replace with your
real bank/crypto details,
# or wire this up to Phase 2
gateway integrations later.
PAYMENT_INSTRUCTIONS =
os.getenv(
    "PAYMENT_INSTRUCTIONS",
    "Bank Transfer:\n"
    "  Bank: <your bank>\n"
    "  Account Name: <your
name>\n"
    "  Account Number: <your
number>\n\n"
    "Or Crypto (USDT TRC20 / BTC
/ ETH / BNB):\n"
    "  Wallet: <your wallet
address>\n\n"
    "After paying, send a
screenshot of the payment here."
)
```

```
def is_admin(user_id: int) ->
bool:
    return user_id in ADMIN_IDS


#
================================
=========================
# DATABASE
#
================================
=========================

SCHEMA = """
CREATE TABLE IF NOT EXISTS users
(
    user_id     INTEGER PRIMARY
KEY,
    username    TEXT,
    full_name   TEXT,
    role        TEXT NOT NULL
DEFAULT 'buyer',
    seller_status TEXT DEFAULT
NULL,
```

```
    shop_name   TEXT,
    created_at  TEXT DEFAULT
CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS
categories (
    category_id INTEGER PRIMARY
KEY AUTOINCREMENT,
    name        TEXT NOT NULL
UNIQUE,
    emoji       TEXT DEFAULT
'📦'
);

CREATE TABLE IF NOT EXISTS
products (
    product_id   INTEGER PRIMARY
KEY AUTOINCREMENT,
    seller_id    INTEGER NOT
NULL,
    category_id  INTEGER,
    title        TEXT NOT NULL,
    description  TEXT,
    price        REAL NOT NULL
```

```
DEFAULT 0,
    stock_qty    INTEGER NOT
NULL DEFAULT 0,
    status       TEXT NOT NULL
DEFAULT 'active',
    created_at   TEXT DEFAULT
CURRENT_TIMESTAMP,
    FOREIGN KEY (seller_id)
REFERENCES users(user_id),
    FOREIGN KEY (category_id)
REFERENCES
categories(category_id)
);

CREATE TABLE IF NOT EXISTS
product_photos (
    photo_id     INTEGER PRIMARY
KEY AUTOINCREMENT,
    product_id   INTEGER NOT
NULL,
    file_id      TEXT NOT NULL,
    FOREIGN KEY (product_id)
REFERENCES products(product_id)
ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS
favourites (
    user_id     INTEGER NOT
NULL,
    product_id  INTEGER NOT
NULL,
    PRIMARY KEY (user_id,
product_id),
    FOREIGN KEY (user_id)
REFERENCES users(user_id),
    FOREIGN KEY (product_id)
REFERENCES products(product_id)
ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS
orders (
    order_id      INTEGER
PRIMARY KEY AUTOINCREMENT,
    buyer_id      INTEGER NOT
NULL,
    product_id    INTEGER NOT
NULL,
    seller_id     INTEGER NOT
```

```
NULL,
    quantity      INTEGER NOT
NULL DEFAULT 1,
    total_price   REAL NOT NULL
DEFAULT 0,
    status        TEXT NOT NULL
DEFAULT 'awaiting_payment',
    -- status: awaiting_payment
| awaiting_approval | approved |
rejected | completed
    screenshot_file_id TEXT,
    created_at    TEXT DEFAULT
CURRENT_TIMESTAMP,
    FOREIGN KEY (buyer_id)
REFERENCES users(user_id),
    FOREIGN KEY (product_id)
REFERENCES products(product_id),
    FOREIGN KEY (seller_id)
REFERENCES users(user_id)
);

CREATE TABLE IF NOT EXISTS
reviews (
    review_id   INTEGER PRIMARY
KEY AUTOINCREMENT,
```

```
    order_id    INTEGER NOT
NULL,
    buyer_id    INTEGER NOT
NULL,
    product_id  INTEGER NOT
NULL,
    rating      INTEGER NOT
NULL,
    comment     TEXT,
    created_at  TEXT DEFAULT
CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id)
REFERENCES orders(order_id),
    FOREIGN KEY (buyer_id)
REFERENCES users(user_id),
    FOREIGN KEY (product_id)
REFERENCES products(product_id)
);
"""


async def init_db():
    async with
aiosqlite.connect(DB_PATH) as
db:
```

```
        await
db.executescript(SCHEMA)
        await db.commit()


async def upsert_user(user_id:
int, username: str, full_name:
str):
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            """INSERT INTO users
(user_id, username, full_name)
               VALUES (?, ?, ?)
               ON
CONFLICT(user_id) DO UPDATE SET
username=excluded.username,
full_name=excluded.full_name""",
            (user_id, username,
full_name),
        )
        await db.commit()
```

```
async def get_user(user_id:
int):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await
db.execute("SELECT * FROM users
WHERE user_id = ?", (user_id,))
        return await
cur.fetchone()


async def
request_seller_status(user_id:
int, shop_name: str):
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            "UPDATE users SET
seller_status='pending',
shop_name=? WHERE user_id=?",
            (shop_name,
```

```
user_id),
        )
        await db.commit()


async def
set_seller_decision(user_id:
int, approved: bool):
    status = "approved" if
approved else "rejected"
    role = "seller" if approved
else "buyer"
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            "UPDATE users SET
seller_status=?, role=? WHERE
user_id=?",
            (status, role,
user_id),
        )
        await db.commit()
```

```
async def get_pending_sellers():
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await
db.execute("SELECT * FROM users
WHERE seller_status='pending'")
        return await
cur.fetchall()


async def add_category(name:
str, emoji: str = "📦"):
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            "INSERT OR IGNORE
INTO categories (name, emoji)
VALUES (?, ?)", (name, emoji)
        )
        await db.commit()
```

```
async def list_categories():
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await
db.execute("SELECT * FROM
categories ORDER BY name")
        return await
cur.fetchall()


async def
create_product(seller_id: int,
category_id: int, title: str,

description: str, price: float,
stock_qty: int) -> int:
    async with
aiosqlite.connect(DB_PATH) as
db:
        cur = await db.execute(
            """INSERT INTO
```

```
products (seller_id,
category_id, title, description,
price, stock_qty)
               VALUES (?, ?, ?,
?, ?, ?)""",
            (seller_id,
category_id, title, description,
price, stock_qty),
        )
        await db.commit()
        return cur.lastrowid


async def
get_product(product_id: int):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await
db.execute("SELECT * FROM
products WHERE product_id=?",
(product_id,))
        return await
```

```
cur.fetchone()


async def
add_product_photo(product_id:
int, file_id: str):
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            "INSERT INTO
product_photos (product_id,
file_id) VALUES (?, ?)",
            (product_id,
file_id),
        )
        await db.commit()


async def
get_product_photos(product_id:
int):
    async with
aiosqlite.connect(DB_PATH) as
db:
```

```
        db.row_factory =
aiosqlite.Row
        cur = await db.execute(
            "SELECT * FROM
product_photos WHERE
product_id=?", (product_id,)
        )
        return await
cur.fetchall()


async def
list_products_by_category(catego
ry_id: int, limit: int = 20,
offset: int = 0):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await db.execute(
            """SELECT * FROM
products WHERE category_id=? AND
status='active'
               ORDER BY
```

```
created_at DESC LIMIT ? OFFSET
?""",
            (category_id, limit,
offset),
        )
        return await
cur.fetchall()


async def
list_products_by_seller(seller_i
d: int):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await db.execute(
            "SELECT * FROM
products WHERE seller_id=? ORDER
BY created_at DESC",
(seller_id,)
        )
        return await
cur.fetchall()
```

```
async def search_products(query:
str, limit: int = 20):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        like = f"%{query}%"
        cur = await db.execute(
            """SELECT * FROM
products
               WHERE
status='active' AND (title LIKE
? OR description LIKE ?)
               ORDER BY
created_at DESC LIMIT ?""",
            (like, like, limit),
        )
        return await
cur.fetchall()


async def
```

```
toggle_favourite(user_id: int,
product_id: int) -> bool:
    async with
aiosqlite.connect(DB_PATH) as
db:
        cur = await db.execute(
            "SELECT 1 FROM
favourites WHERE user_id=? AND
product_id=?", (user_id,
product_id)
        )
        exists = await
cur.fetchone()
        if exists:
            await db.execute(
                "DELETE FROM
favourites WHERE user_id=? AND
product_id=?", (user_id,
product_id)
            )
            await db.commit()
            return False
        else:
            await db.execute(
                "INSERT INTO
```

```
favourites (user_id, product_id)
VALUES (?, ?)", (user_id,
product_id)
            )
            await db.commit()
            return True


async def
list_favourites(user_id: int):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await db.execute(
            """SELECT p.* FROM
favourites f
               JOIN products p
ON p.product_id = f.product_id
               WHERE f.user_id=?
ORDER BY p.created_at DESC""",
            (user_id,),
        )
        return await
```

```
cur.fetchall()


# ---------- Orders ----------

async def create_order(buyer_id:
int, product_id: int, seller_id:
int,

quantity: int, total_price:
float) -> int:
    async with
aiosqlite.connect(DB_PATH) as
db:
        cur = await db.execute(
            """INSERT INTO
orders (buyer_id, product_id,
seller_id, quantity,
total_price, status)
               VALUES (?, ?, ?,
?, ?, 'awaiting_payment')""",
            (buyer_id,
product_id, seller_id, quantity,
total_price),
        )
```

```
        await db.commit()
        return cur.lastrowid


async def
attach_order_screenshot(order_id
: int, file_id: str):
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            "UPDATE orders SET
screenshot_file_id=?,
status='awaiting_approval' WHERE
order_id=?",
            (file_id, order_id),
        )
        await db.commit()


async def get_order(order_id:
int):
    async with
aiosqlite.connect(DB_PATH) as
db:
```

```
        db.row_factory =
aiosqlite.Row
        cur = await
db.execute("SELECT * FROM orders
WHERE order_id=?", (order_id,))
        return await
cur.fetchone()


async def
set_order_status(order_id: int,
status: str):
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute("UPDATE
orders SET status=? WHERE
order_id=?", (status, order_id))
        await db.commit()


async def
decrement_stock(product_id: int,
quantity: int):
    async with
```

```
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            "UPDATE products SET
stock_qty = MAX(0, stock_qty -
?) WHERE product_id=?",
            (quantity,
product_id),
        )
        await db.commit()


async def
list_orders_by_buyer(buyer_id:
int):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await db.execute(
            "SELECT * FROM
orders WHERE buyer_id=? ORDER BY
created_at DESC", (buyer_id,)
        )
```

```
        return await
cur.fetchall()


async def
list_orders_for_seller(seller_id
: int, status: str = None):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        if status:
            cur = await
db.execute(
                "SELECT * FROM
orders WHERE seller_id=? AND
status=? ORDER BY created_at
DESC",
                (seller_id,
status),
            )
        else:
            cur = await
db.execute(
```

```
                "SELECT * FROM
orders WHERE seller_id=? ORDER
BY created_at DESC",
(seller_id,)
            )
        return await
cur.fetchall()


async def
list_orders_awaiting_approval():
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await db.execute(
            "SELECT * FROM
orders WHERE
status='awaiting_approval' ORDER
BY created_at ASC"
        )
        return await
cur.fetchall()
```

```
# ---------- Reviews ----------

async def add_review(order_id:
int, buyer_id: int, product_id:
int, rating: int, comment: str):
    async with
aiosqlite.connect(DB_PATH) as
db:
        await db.execute(
            """INSERT INTO
reviews (order_id, buyer_id,
product_id, rating, comment)
               VALUES (?, ?, ?,
?, ?)""",
            (order_id, buyer_id,
product_id, rating, comment),
        )
        await db.commit()


async def has_review(order_id:
int) -> bool:
    async with
aiosqlite.connect(DB_PATH) as
```

```
db:
        cur = await
db.execute("SELECT 1 FROM
reviews WHERE order_id=?",
(order_id,))
        return (await
cur.fetchone()) is not None


async def
list_reviews_for_product(product
_id: int):
    async with
aiosqlite.connect(DB_PATH) as
db:
        db.row_factory =
aiosqlite.Row
        cur = await db.execute(
            "SELECT * FROM
reviews WHERE product_id=? ORDER
BY created_at DESC",
(product_id,)
        )
        return await
cur.fetchall()
```

```
#
================================
=========================
# KEYBOARDS
#
================================
=========================

def main_menu(is_seller: bool,
is_admin_user: bool) ->
ReplyKeyboardMarkup:
    rows = [
        [KeyboardButton(text="🔎
Search"),
KeyboardButton(text="📂
Categories")],
        [KeyboardButton(text="⭐
Favourites"),
KeyboardButton(text="📦 My
Orders")],
        [KeyboardButton(text="👤
My Account")],
    ]
```

```
    if is_seller:

rows.append([KeyboardButton(text
="🛍️ My Products"),
KeyboardButton(text="➕ Add
Product")])

rows.append([KeyboardButton(text
="📥 Orders to Fulfil")])
    else:

rows.append([KeyboardButton(text
="🏪 Become a Seller")])
    if is_admin_user:

rows.append([KeyboardButton(text
="🛠️ Admin Panel")])
    return
ReplyKeyboardMarkup(keyboard=row
s, resize_keyboard=True)


def categories_kb(categories) ->
InlineKeyboardMarkup:
    buttons = [
```

```
[InlineKeyboardButton(text=f"
{c['emoji']} {c['name']}",
callback_data=f"cat:
{c['category_id']}")]
        for c in categories
    ]
    return
InlineKeyboardMarkup(inline_keyb
oard=buttons)


def product_card_kb(product_id:
int, is_fav: bool) ->
InlineKeyboardMarkup:
    fav_label = "💔 Remove
favourite" if is_fav else "❤️
Add to favourites"
    return
InlineKeyboardMarkup(inline_keyb
oard=[

[InlineKeyboardButton(text="🛒
Buy", callback_data=f"buy:
{product_id}")],
```

```
[InlineKeyboardButton(text=fav_l
abel, callback_data=f"fav:
{product_id}")],
    ])


def order_approval_kb(order_id:
int) -> InlineKeyboardMarkup:
    return
InlineKeyboardMarkup(inline_keyb
oard=[
        [

InlineKeyboardButton(text="✅
Approve",
callback_data=f"order_ok:
{order_id}"),

InlineKeyboardButton(text="❌
Reject",
callback_data=f"order_no:
{order_id}"),
        ]
    ])
```

```
def review_rating_kb(order_id:
int) -> InlineKeyboardMarkup:
    return
InlineKeyboardMarkup(inline_keyb
oard=[

[InlineKeyboardButton(text="⭐"
* n, callback_data=f"rate:
{order_id}:{n}") for n in
range(1, 4)],

[InlineKeyboardButton(text="⭐"
* n, callback_data=f"rate:
{order_id}:{n}") for n in
range(4, 6)],
    ])


def admin_panel_kb() ->
InlineKeyboardMarkup:
    return
InlineKeyboardMarkup(inline_keyb
oard=[
```

```
[InlineKeyboardButton(text="👥
Pending Sellers",
callback_data="admin:pending_sel
lers")],

[InlineKeyboardButton(text="📂
Add Category",
callback_data="admin:add_categor
y")],

[InlineKeyboardButton(text="📊
Stats",
callback_data="admin:stats")],
    ])


def seller_decision_kb(user_id:
int) -> InlineKeyboardMarkup:
    return
InlineKeyboardMarkup(inline_keyb
oard=[
        [

InlineKeyboardButton(text="✅
```

```
Approve",
callback_data=f"seller_ok:
{user_id}"),

InlineKeyboardButton(text="❌
Reject",
callback_data=f"seller_no:
{user_id}"),
        ]
    ])


#
================================
========================
# FSM STATES
#
================================
========================

class BecomeSeller(StatesGroup):
    waiting_shop_name = State()


class AddProduct(StatesGroup):
```

```
    choosing_category = State()
    waiting_title = State()
    waiting_description =
State()
    waiting_price = State()
    waiting_stock = State()
    waiting_photos = State()


class AddCategory(StatesGroup):
    waiting_name = State()


class Search(StatesGroup):
    waiting_query = State()


class BuyProduct(StatesGroup):
    waiting_quantity = State()
    waiting_screenshot = State()


class LeaveReview(StatesGroup):
    waiting_comment = State()
```

```
#
================================
========================
# ROUTER: START / ACCOUNT
#
================================
========================

start_router = Router()


async def _menu_for(user_id:
int):
    user = await
get_user(user_id)
    return main_menu(is_seller=
(user["role"] == "seller"),
is_admin_user=is_admin(user_id))


@start_router.message(CommandSta
rt())
async def cmd_start(message:
Message):
```

```
    await
upsert_user(message.from_user.id
, message.from_user.username or
"", message.from_user.full_name
or "")
    menu = await
_menu_for(message.from_user.id)
    await message.answer(
        "👋 Welcome to the
Marketplace!\n\nBrowse products,
save favourites, or sell your
own items.",
        reply_markup=menu,
    )


@start_router.message(F.text ==
"👤 My Account")
async def my_account(message:
Message):
    user = await
get_user(message.from_user.id)
    lines = [f"👤 Role:
{user['role'].capitalize()}"]
    if user["seller_status"] ==
```

```
"pending":
        lines.append("🕓 Seller
application: pending approval")
    elif user["seller_status"]
== "rejected":
        lines.append("❌ Seller
application: rejected")
    if user["shop_name"]:
        lines.append(f"🏪 Shop:
{user['shop_name']}")
    await
message.answer("\n".join(lines))


@start_router.message(F.text ==
"🏪 Become a Seller")
async def
become_seller_start(message:
Message, state: FSMContext):
    user = await
get_user(message.from_user.id)
    if user["role"] == "seller":
        await
message.answer("You're already
an approved seller ✅")
```

```
        return
    if user["seller_status"] ==
"pending":
        await
message.answer("Your seller
application is already pending
review 🕓")
        return
    await message.answer("What's
your shop name?")
    await
state.set_state(BecomeSeller.wai
ting_shop_name)


@start_router.message(BecomeSell
er.waiting_shop_name)
async def
become_seller_finish(message:
Message, state: FSMContext, bot:
Bot):
    shop_name =
message.text.strip()
    await
request_seller_status(message.fr
```

```
om_user.id, shop_name)
    await state.clear()
    await message.answer("✅
Application submitted! We'll
notify you once an admin reviews
it.")
    for admin_id in ADMIN_IDS:
        try:
            await
bot.send_message(
                admin_id,
                f"🆕 Seller
application\nUser:
{message.from_user.full_name}
(@{message.from_user.username})\
n"
                f"Shop name:
{shop_name}",

reply_markup=seller_decision_kb(
message.from_user.id),
            )
        except Exception:
            pass
```

```
#
================================
========================
# ROUTER: SELLER
#
================================
========================

seller_router = Router()


async def
_require_seller(message:
Message) -> bool:
    user = await
get_user(message.from_user.id)
    if user["role"] != "seller":
        await
message.answer("Only approved
sellers can do this. Tap '🏪
Become a Seller' first.")
        return False
    return True
```

```
@seller_router.message(F.text ==
"➕ Add Product")
async def
add_product_start(message:
Message, state: FSMContext):
    if not await
_require_seller(message):
        return
    categories = await
list_categories()
    if not categories:
        await message.answer("No
categories exist yet — ask an
admin to add one first.")
        return
    await
state.set_state(AddProduct.choos
ing_category)
    await message.answer("Choose
a category for your product:",
reply_markup=categories_kb(categ
ories))
```

```
@seller_router.callback_query(AddProduct.choosing_category, F.data.startswith("cat:"))
async def add_product_category_chosen(callback: CallbackQuery, state: FSMContext):
    category_id = int(callback.data.split(":")[1])
    await state.update_data(category_id=category_id)
    await state.set_state(AddProduct.waiting_title)
    await callback.message.answer("What's the product title?")
    await callback.answer()


@seller_router.message(AddProduct.waiting_title)
async def add_product_title(message:
```

```
Message, state: FSMContext):
    await
state.update_data(title=message.
text.strip())
    await
state.set_state(AddProduct.waiti
ng_description)
    await message.answer("Give a
short description:")


@seller_router.message(AddProduc
t.waiting_description)
async def
add_product_description(message:
Message, state: FSMContext):
    await
state.update_data(description=me
ssage.text.strip())
    await
state.set_state(AddProduct.waiti
ng_price)
    await message.answer("Price
(numbers only, e.g. 2500):")
```

```
@seller_router.message(AddProduc
t.waiting_price)
async def
add_product_price(message:
Message, state: FSMContext):
    try:
        price =
float(message.text.strip())
    except ValueError:
        await
message.answer("Please send a
valid number for price.")
        return
    await
state.update_data(price=price)
    await
state.set_state(AddProduct.waiti
ng_stock)
    await message.answer("Stock
quantity available:")


@seller_router.message(AddProduc
t.waiting_stock)
```

```
async def
add_product_stock(message:
Message, state: FSMContext):
    try:
        stock =
int(message.text.strip())
    except ValueError:
        await
message.answer("Please send a
whole number for stock
quantity.")
        return
    data = await
state.get_data()
    product_id = await
create_product(

seller_id=message.from_user.id,

category_id=data["category_id"],
        title=data["title"],

description=data["description"],
        price=data["price"],
        stock_qty=stock,
```

```
    )
    await
state.update_data(product_id=pro
duct_id, photo_count=0)
    await
state.set_state(AddProduct.waiti
ng_photos)
    await message.answer("Now
send product photos, one at a
time. Send /done when finished
(at least 1 photo).")


@seller_router.message(AddProduc
t.waiting_photos, F.photo)
async def
add_product_photo_handler(messag
e: Message, state: FSMContext):
    data = await
state.get_data()
    file_id =
message.photo[-1].file_id
    await
add_product_photo(data["product_
id"], file_id)
```

```
    count =
data.get("photo_count", 0) + 1
    await
state.update_data(photo_count=co
unt)
    await message.answer(f"📸
Photo {count} saved. Send
another or /done to finish.")


@seller_router.message(AddProduc
t.waiting_photos, F.text ==
"/done")
async def
add_product_finish(message:
Message, state: FSMContext):
    data = await
state.get_data()
    if data.get("photo_count",
0) == 0:
        await
message.answer("Please send at
least one photo before
finishing.")
        return
```

```
    await state.clear()
    await message.answer(f"✅
Product '{data['title']}'
published!")


@seller_router.message(F.text ==
"🛍️ My Products")
async def my_products(message:
Message):
    if not await
_require_seller(message):
        return
    products = await
list_products_by_seller(message.
from_user.id)
    if not products:
        await
message.answer("You haven't
listed any products yet.")
        return
    for p in products:
        await message.answer(
            f"🛍️ {p['title']}\n
💰 ₦{p['price']:.2f}\n📦 Stock:
```

```
{p['stock_qty']}\n"
            f"Status:
{p['status']}\nID:
{p['product_id']}"
        )


#
================================
=========================
# ROUTER: BUYER (browse / search
/ favourites)
#
================================
=========================

buyer_router = Router()


async def
_send_product_card(target_messag
e: Message, product, user_id:
int):
    photos = await
get_product_photos(product["prod
```

```
uct_id"])
    favs = await
list_favourites(user_id)
    is_fav = any(f["product_id"]
== product["product_id"] for f
in favs)
    caption = (
        f"🛍️ <b>
{product['title']}
</b>\n{product['description']}\n
\n"
        f"💰 ₦
{product['price']:.2f}\n📦
Stock: {product['stock_qty']}"
    )
    markup =
product_card_kb(product["product
_id"], is_fav)
    if photos:
        await
target_message.answer_photo(phot
os[0]["file_id"],
caption=caption,
parse_mode="HTML",
reply_markup=markup)
```

```
    else:
        await
target_message.answer(caption,
parse_mode="HTML",
reply_markup=markup)


@buyer_router.message(F.text ==
"📂 Categories")
async def
show_categories(message:
Message):
    categories = await
list_categories()
    if not categories:
        await message.answer("No
categories yet.")
        return
    await message.answer("Pick a
category:",
reply_markup=categories_kb(categ
ories))


@buyer_router.callback_query(F.d
```

```
ata.startswith("cat:"))
async def
browse_category(callback:
CallbackQuery):
    category_id =
int(callback.data.split(":")[1])
    products = await
list_products_by_category(catego
ry_id)
    await callback.answer()
    if not products:
        await
callback.message.answer("No
products in this category yet.")
        return
    for p in products:
        await
_send_product_card(callback.mess
age, p, callback.from_user.id)


@buyer_router.message(F.text ==
"🔎 Search")
async def search_start(message:
Message, state: FSMContext):
```

```
    await
state.set_state(Search.waiting_q
uery)
    await message.answer("What
are you looking for?")


@buyer_router.message(Search.wai
ting_query)
async def search_run(message:
Message, state: FSMContext):
    await state.clear()
    results = await
search_products(message.text.str
ip())
    if not results:
        await message.answer("No
matching products found.")
        return
    for p in results:
        await
_send_product_card(message, p,
message.from_user.id)
```

```
@buyer_router.message(F.text ==
"⭐ Favourites")
async def
show_favourites(message:
Message):
    favs = await
list_favourites(message.from_use
r.id)
    if not favs:
        await
message.answer("You have no
favourites yet.")
        return
    for p in favs:
        await
_send_product_card(message, p,
message.from_user.id)


@buyer_router.callback_query(F.d
ata.startswith("fav:"))
async def
toggle_fav_handler(callback:
CallbackQuery):
    product_id =
```

```
int(callback.data.split(":")[1])
    now_fav = await
toggle_favourite(callback.from_u
ser.id, product_id)
    await callback.answer("Added
to favourites ❤️" if now_fav
else "Removed from favourites
💔")


#
================================
=========================
# ROUTER: ORDERS (Phase 4)
#
================================
=========================

orders_router = Router()


@orders_router.callback_query(F.
data.startswith("buy:"))
async def buy_start(callback:
CallbackQuery, state:
```

```
FSMContext):
    product_id =
int(callback.data.split(":")[1])
    product = await
get_product(product_id)
    await callback.answer()
    if not product or
product["status"] != "active":
        await
callback.message.answer("This
product isn't available
anymore.")
        return
    if product["stock_qty"] <=
0:
        await
callback.message.answer("This
product is out of stock.")
        return
    if product["seller_id"] ==
callback.from_user.id:
        await
callback.message.answer("You
can't buy your own product.")
        return
```

```
    await
state.update_data(product_id=pro
duct_id)
    await
state.set_state(BuyProduct.waiti
ng_quantity)
    await
callback.message.answer(
        f"How many units of
'{product['title']}' would you
like? (in stock:
{product['stock_qty']})"
    )


@orders_router.message(BuyProduc
t.waiting_quantity)
async def buy_quantity(message:
Message, state: FSMContext):
    try:
        qty =
int(message.text.strip())
        if qty <= 0:
            raise ValueError
    except ValueError:
```

```
        await
message.answer("Please send a
valid positive whole number.")
        return

    data = await
state.get_data()
    product = await
get_product(data["product_id"])
    if not product or
product["stock_qty"] < qty:
        await
message.answer("Sorry, that's
more than what's in stock. Try a
smaller quantity.")
        return

    total = product["price"] *
qty
    order_id = await
create_order(

buyer_id=message.from_user.id,

product_id=product["product_id"]
```

```
,

seller_id=product["seller_id"],
        quantity=qty,
        total_price=total,
    )
    await
state.update_data(order_id=order
_id)
    await
state.set_state(BuyProduct.waiti
ng_screenshot)
    await message.answer(
        f"🧾 Order #{order_id}
created — {qty} ×
{product['title']} = ₦
{total:.2f}\n\n"
        f"
{PAYMENT_INSTRUCTIONS}"
    )


@orders_router.message(BuyProduc
t.waiting_screenshot, F.photo)
async def
```

```
buy_screenshot(message: Message,
state: FSMContext, bot: Bot):
    data = await
state.get_data()
    order_id = data["order_id"]
    file_id =
message.photo[-1].file_id
    await
attach_order_screenshot(order_id
, file_id)
    order = await
get_order(order_id)
    product = await
get_product(order["product_id"])
    await state.clear()
    await message.answer(
        f"✅ Payment screenshot
received for order #{order_id}.
"
        f"Waiting for
seller/admin approval — you'll
be notified."
    )

    notify_text = (
```

```
        f"💰 New payment to
review — Order #{order_id}\n"
        f"Product:
{product['title']}\n"
        f"Qty:
{order['quantity']} — Total: ₦
{order['total_price']:.2f}\n"
        f"Buyer:
{message.from_user.full_name}
(@{message.from_user.username})"
    )
    recipients =
{order["seller_id"], *ADMIN_IDS}
    for uid in recipients:
        try:
            await
bot.send_photo(
                uid, file_id,
caption=notify_text,
reply_markup=order_approval_kb(o
rder_id)
            )
        except Exception:
            pass
```

```
@orders_router.message(BuyProduct.waiting_screenshot)
async def buy_screenshot_wrong_type(message: Message):
    await message.answer("Please send a photo of your payment receipt/screenshot.")


@orders_router.message(F.text == "📦 My Orders")
async def my_orders(message: Message):
    orders = await list_orders_by_buyer(message.from_user.id)
    if not orders:
        await message.answer("You haven't placed any orders yet.")
        return
    for o in orders:
        product = await
```

```
get_product(o["product_id"])
        title = product["title"]
if product else "(deleted
product)"
        text = (
            f"🧾 Order #
{o['order_id']} — {title}\n"
            f"Qty:
{o['quantity']} — Total: ₦
{o['total_price']:.2f}\n"
            f"Status:
{o['status'].replace('_', '
').title()}"
        )
        if o["status"] ==
"completed" and not await
has_review(o["order_id"]):
            await
message.answer(text,
reply_markup=review_rating_kb(o[
"order_id"]))
        else:
            await
message.answer(text)
```

```
@orders_router.message(F.text ==
"📥 Orders to Fulfil")
async def
orders_to_fulfil(message:
Message):
    user = await
get_user(message.from_user.id)
    if user["role"] != "seller":
        await
message.answer("Only approved
sellers can view this.")
        return
    orders = await
list_orders_for_seller(message.f
rom_user.id,
status="awaiting_approval")
    if not orders:
        await message.answer("No
orders awaiting your approval
right now.")
        return
    for o in orders:
        product = await
get_product(o["product_id"])
```

```
        text = (
            f"🧾 Order #
{o['order_id']} —
{product['title'] if product
else '(deleted)'}\n"
            f"Qty:
{o['quantity']} — Total: ₦
{o['total_price']:.2f}"
        )
        if
o["screenshot_file_id"]:
            await
message.answer_photo(

o["screenshot_file_id"],
caption=text,
reply_markup=order_approval_kb(o
["order_id"])
            )
        else:
            await
message.answer(text,
reply_markup=order_approval_kb(o
["order_id"]))
```

```
async def
_can_manage_order(user_id: int,
order) -> bool:
    return is_admin(user_id) or
user_id == order["seller_id"]


@orders_router.callback_query(F.
data.startswith("order_ok:"))
async def
approve_order(callback:
CallbackQuery, bot: Bot):
    order_id =
int(callback.data.split(":")[1])
    order = await
get_order(order_id)
    if not order:
        await
callback.answer("Order not
found.")
        return
    if not await
_can_manage_order(callback.from_
user.id, order):
```

```
        await
callback.answer("Not
authorized.")
        return
    if order["status"] !=
"awaiting_approval":
        await
callback.answer("Already
handled.")
        return

    await
decrement_stock(order["product_i
d"], order["quantity"])
    await
set_order_status(order_id,
"completed")
    await
callback.answer("Approved ✅")
    await
callback.message.answer(f"Order
#{order_id} approved and marked
completed.")

    product = await
```

```
get_product(order["product_id"])
    try:
        await bot.send_message(
            order["buyer_id"],
            f"🎉 Your payment
for order #{order_id}
({product['title'] if product
else ''}) was approved!\n"
            f"How would you rate
this purchase?",

reply_markup=review_rating_kb(or
der_id),
        )
    except Exception:
        pass


@orders_router.callback_query(F.
data.startswith("order_no:"))
async def reject_order(callback:
CallbackQuery, bot: Bot):
    order_id =
int(callback.data.split(":")[1])
    order = await
```

```
get_order(order_id)
    if not order:
        await
callback.answer("Order not
found.")
        return
    if not await
_can_manage_order(callback.from_
user.id, order):
        await
callback.answer("Not
authorized.")
        return
    if order["status"] !=
"awaiting_approval":
        await
callback.answer("Already
handled.")
        return

    await
set_order_status(order_id,
"rejected")
    await
callback.answer("Rejected ❌")
```

```
    await
callback.message.answer(f"Order
#{order_id} rejected.")
    try:
        await bot.send_message(
            order["buyer_id"],
            f"❌ Your payment
for order #{order_id} could not
be verified. "
            f"Please contact
support or try again."
        )
    except Exception:
        pass


@orders_router.callback_query(F.
data.startswith("rate:"))
async def rate_order(callback:
CallbackQuery, state:
FSMContext):
    _, order_id_str, rating_str
= callback.data.split(":")
    order_id, rating =
int(order_id_str),
```

```
int(rating_str)
    order = await
get_order(order_id)
    await callback.answer()
    if not order or
order["buyer_id"] !=
callback.from_user.id:
        return
    if await
has_review(order_id):
        await
callback.message.answer("You've
already reviewed this order.")
        return
    await
state.update_data(order_id=order
_id,
product_id=order["product_id"],
rating=rating)
    await
state.set_state(LeaveReview.wait
ing_comment)
    await
callback.message.answer("Thanks!
Add a short comment for your
```

```
review (or send '-' to skip):")


@orders_router.message(LeaveRevi
ew.waiting_comment)
async def
rate_order_comment(message:
Message, state: FSMContext):
    data = await
state.get_data()
    comment = "" if
message.text.strip() == "-" else
message.text.strip()
    await
add_review(data["order_id"],
message.from_user.id,
data["product_id"],
data["rating"], comment)
    await state.clear()
    await message.answer(f"✅
Review submitted: {'⭐' *
data['rating']}")


#
```

```
================================
========================
# ROUTER: ADMIN
#
================================
========================

admin_router = Router()


@admin_router.message(F.text ==
"🛠️ Admin Panel")
async def admin_panel(message:
Message):
    if not
is_admin(message.from_user.id):
        return
    await message.answer("Admin
Panel:",
reply_markup=admin_panel_kb())


@admin_router.callback_query(F.d
ata == "admin:pending_sellers")
async def
```

```
pending_sellers(callback:
CallbackQuery):
    if not
is_admin(callback.from_user.id):
        await callback.answer()
        return
    pending = await
get_pending_sellers()
    await callback.answer()
    if not pending:
        await
callback.message.answer("No
pending seller applications.")
        return
    for u in pending:
        await
callback.message.answer(
            f"👤
{u['full_name']}
(@{u['username']})\n🏪 Shop:
{u['shop_name']}",

reply_markup=seller_decision_kb(
u["user_id"]),
        )
```

```
@admin_router.callback_query(F.d
ata.startswith("seller_ok:"))
async def
approve_seller(callback:
CallbackQuery, bot: Bot):
    if not
is_admin(callback.from_user.id):
        await callback.answer()
        return
    user_id =
int(callback.data.split(":")[1])
    await
set_seller_decision(user_id,
approved=True)
    await
callback.answer("Approved ✅")
    await
callback.message.answer(f"Seller
{user_id} approved.")
    try:
        await
bot.send_message(user_id, "🎉
Your seller application was
```

```
approved! You can now add
products.")
    except Exception:
        pass


@admin_router.callback_query(F.d
ata.startswith("seller_no:"))
async def
reject_seller(callback:
CallbackQuery, bot: Bot):
    if not
is_admin(callback.from_user.id):
        await callback.answer()
        return
    user_id =
int(callback.data.split(":")[1])
    await
set_seller_decision(user_id,
approved=False)
    await
callback.answer("Rejected ❌")
    await
callback.message.answer(f"Seller
{user_id} rejected.")
```

```
    try:
        await
bot.send_message(user_id, "Your
seller application was not
approved this time.")
    except Exception:
        pass


@admin_router.callback_query(F.d
ata == "admin:add_category")
async def
add_category_start(callback:
CallbackQuery, state:
FSMContext):
    if not
is_admin(callback.from_user.id):
        await callback.answer()
        return
    await callback.answer()
    await
state.set_state(AddCategory.wait
ing_name)
    await
callback.message.answer("Send
```

```
the category name (optionally
start with an emoji, e.g. '📱
Phones'):")


@admin_router.message(AddCategor
y.waiting_name)
async def
add_category_finish(message:
Message, state: FSMContext):
    text = message.text.strip()
    parts = text.split(" ", 1)
    if len(parts) == 2 and
len(parts[0]) <= 2:
        emoji, name = parts[0],
parts[1]
    else:
        emoji, name = "📦", text
    await add_category(name,
emoji)
    await state.clear()
    await message.answer(f"✅
Category added: {emoji} {name}")
```

```
@admin_router.callback_query(F.d
ata == "admin:stats")
async def stats(callback:
CallbackQuery):
    if not
is_admin(callback.from_user.id):
        await callback.answer()
        return
    await callback.answer()
    categories = await
list_categories()
    await
callback.message.answer(f"📊
Categories:
{len(categories)}\n(More stats
land in Phase 5.)")


#
================================
=========================
# ENTRYPOINT
#
================================
=========================
```

```
async def main():
    await init_db()

    bot = Bot(token=BOT_TOKEN,
default=DefaultBotProperties(par
se_mode="HTML"))
    dp =
Dispatcher(storage=MemoryStorage
())


dp.include_router(start_router)

dp.include_router(seller_router)

dp.include_router(admin_router)

dp.include_router(buyer_router)

dp.include_router(orders_router)

    print("Bot is starting...")
    await dp.start_polling(bot)
```

```
if __name__ == "__main__":
    asyncio.run(main())
```